Obradoiro Git e GitHub · Instrucións do Usuario 2

Ti es o **Usuario 2**: únete ao repositorio que crea o teu compañeiro e desenvolve as funcións de clasificación e o listado do informe. Na fase 2 tamén resolverás un conflito. O teu compañeiro ten outras instrucións. Cando vexas 🤝, toca coordinarse con el.

Que necesitas: conta de GitHub, Git, Python 3 e un editor.

Comprobar sempre: `python probas.py` indica se cada función está OK, PENDENTE ou FALLA. Executao antes de cada commit, sempre desde a carpeta raíz do proxecto (non desde `src`).

Fase 0 · Unirte ao repositorio

1. Configura Git (só a primeira vez):

```
git config --global user.name "O teu nome"
git config --global user.email "o-teu-email@exemplo.com"
git config --global pull.rebase false
ssh -T git@github.com    # comproba a chave SSH: debe saudarte co teu usuario
```

2. Dille ao teu compañeiro o teu nome de usuario de GitHub para que te invite.

3. Acepta a invitación: chega por correo ou en github.com/notifications.

> 🤝 **Agarda** a que o teu compañeiro che avise de que os ficheiros xa están en GitHub.

4. Clona o repositorio. Usa a URL SSH (botón verde **Code**, pestana **SSH**):

```
git clone git@github.com:USUARIO-1/proxecto-notas.git
cd proxecto-notas
python probas.py
```

5. Executa `git log --oneline` e compara o resultado co do teu compañeiro: debe ser o mesmo commit.

Fase 1 · O teu ficheiro: `clasificacion.py`

1. Crea a túa rama:

```
git checkout main
git pull
git checkout -b fase1-clasificacion
```

2. Completa `aprobado()`, `contar_aprobados()` e `cualificacion()`. Substitúe cada `raise NotImplementedError` polo teu código.

3. Executa `python probas.py` ata que o bloque "FASE 1 · Usuario 2" estea todo en OK.

4. Garda o traballo, actualiza a rama con `main` e súbea. Nesta fase o normal é que Git responda *Already up to date* ou faga un merge sen conflitos, porque cada un traballa nun ficheiro distinto. Aínda así, faino sempre:

```
git add src/clasificacion.py
git commit -m "Implementa as funcións de clasificación"
git pull origin main
git push -u origin fase1-clasificacion
```

5. En GitHub preme **Compare & pull request**. Comproba: base `main` ← compare `fase1-clasificacion`. Escribe un título e unha descrición, escolle o teu compañeiro en **Reviewers** e preme **Create pull request**.

> 🤝 **Avisa ao teu compañeiro** de que o teu PR está listo. Cando el abra o seu, revísao ti.

6. Abre o PR do compañeiro, pestana **Files changed**. Deixa polo menos un comentario nunha liña (botón **+** ao pasar o rato).
7. Preme **Review changes** → **Approve** → **Submit review** e despois **Merge pull request** → **Confirm merge**.

> 🤝 **Cando os dous PR estean combinados**, actualiza o teu `main`:
>
> ```
> git checkout main
> git pull
> python probas.py
> ```
>
> Toda a fase 1 debe estar en OK. Por que non houbo ningún conflito?

Fase 2 · A túa función: `seccion_listado()`

Nesta fase traballades os dous no mesmo ficheiro, `src/informe.py`.

1. Crea a túa rama partindo do `main` actualizado:

```
git checkout main
git pull
git checkout -b fase2-listado
```

2. Completa `seccion_listado()` seguindo o seu docstring.

3. En `informe_completo()`, xusto debaixo do comentario, engade esta liña:

```
partes.append(seccion_listado(alumnado))
```

4. `python probas.py`: "FASE 2 · Usuario 2" debe estar en OK.
5. **Garda o teu traballo nun commit.** Aínda non fagas push:

```
git add src/informe.py
git commit -m "Engade a sección de listado ao informe"
```

> 🤝 **O teu compañeiro abriu un PR.** Revísao e fai o merge, igual que na fase 1. O teu traballo está seguro no commit do paso 5: aprobar un PR non toca os teus ficheiros.

Resolver o conflito

6. Trae á túa rama o que acaba de entrar en `main`:

```
git pull origin main
```

7. Git avisa: `CONFLICT (content): Merge conflict in src/informe.py`. Abre o ficheiro. A función do teu compañeiro xa apareceu soa, sen conflito. Só hai marcas en `informe_completo()`:

```
<<<<<<< HEAD
    partes.append(seccion_listado(alumnado))
=======
    partes.append(seccion_resumo(alumnado))
>>>>>>> 891654526d48...
```

   A parte de arriba (`HEAD`) é a túa; a de abaixo é a que vén de `main`.

8. Queremos as dúas liñas, primeiro o resumo e despois o listado. Borra as tres liñas de marcas para que quede así:

```
    partes.append(seccion_resumo(alumnado))
    partes.append(seccion_listado(alumnado))
```

   En VS Code tamén podes premer **Accept Both Changes** e logo ordenar as liñas.

9. Comproba: `python probas.py` debe estar todo en OK e `python main.py` debe mostrar o informe completo.
10. Pecha o merge e sube a rama:

```
git add src/informe.py
git commit -m "Resolve conflito con main"
git push -u origin fase2-listado
```

11. Abre o pull request co teu compañeiro como revisor. GitHub debe indicar que non hai conflitos.

> 🤝 **Avisa ao teu compañeiro** para que revise e faga o merge do teu PR.

> 🤝 **Final.** Executade os dous:
>
> ```
> git checkout main
> git pull
> python probas.py
> python main.py
> ```
>
> Todo debe estar en OK e o informe completo debe aparecer en pantalla. Mirade o historial con `git log --oneline --graph`.

Comandos que vas usar

Comando	Para que serve
`git status`	Ver en que rama estás e que cambiou
`git checkout -b nome`	Crear unha rama e cambiar a ela
`git add` / `git commit -m "..."`	Gardar cambios
`git push -u origin nome`	Subir unha rama nova
`git pull`	Traer os cambios de GitHub
`git pull origin main`	Combinar `main` coa túa rama

Regra de ouro: nunca traballes en `main`, e fai commit antes de cambiar de rama ou de traer cambios.